

TFM · MÁSTER DE DESARROLLO CON IA

SprintWell

Planificación de sprints que optimiza el bienestar del equipo

José Carlos Hernández

El problema

**Las tareas se asignan por
capacidad y deadlines.**

**Las personas quedan fuera
del modelo.**

Jira, Linear, Asana... no modelan las preferencias del trabajador.

Cuando se atienden, es ad-hoc — y nadie mide la equidad.

Qué cambia SprintWell

Las **preferencias** pasan a ser un **objetivo de primer orden**, junto a las restricciones operativas y a la **equidad** entre personas.

Antes	Con SprintWell
Capacidad, deadlines, skills	+ preferencias declaradas (DSL de reglas)
Criterio de reparto implícito	Equidad explícita y elegible
Decisión opaca del lead	Plan auditable y explicable

APORTE CENTRAL

NP-hard

No es un tablero que se ordena a mano:
es optimización combinatoria multiobjetivo.

Por qué es NP-hard

Asignar n tareas a m personas **y** decidir el día de inicio de cada una, respetando skills, dependencias, deadlines y capacidad, mientras se maximiza la equidad.

$$\approx m^n$$

combinaciones — el espacio **explota**. No hay fuerza bruta viable.

NP-hard, formalmente

Se demuestra por **reducción desde el Generalized Assignment Problem (GAP)**, que es NP-hard [Fisher et al.].

Consecuencia: salvo que **$P = NP$** , **no existe** un algoritmo exacto en tiempo polinómico.

Esto **no es un tecnicismo**: es lo que **justifica** todo el enfoque algorítmico — hay que usar un solver, no una fórmula.

La respuesta: CP-SAT

Un solver **heurístico-completo** (Google OR-Tools):

- Prueba **optimalidad** si dispone de tiempo...
- ...y devuelve **soluciones de calidad** dentro de un presupuesto acotado (uso interactivo).

Las preferencias (DSL de 12 reglas) se compilan a restricciones; la equidad se agrega en **3 modos**: utilitarista · max-min · Nash.

Arquitectura — 3 servicios

Servicio	Rol	Stack
Backend	dominio + API	NestJS · DDD 4 capas · Prisma · PostgreSQL
Optimizer	resuelve el NP-hard	Python · FastAPI · OR-Tools CP-SAT
Frontend	UI · 3 perfiles	React · Vite · Tailwind · cliente tipado desde OpenAPI

DEMO EN VIVO



Verlo funcionando

login → sprint + tareas → **editor de reglas**
→ **planificar (Nash)** → Gantt + bienestar
→ **comparar Nash vs Utilitario** → explicabilidad

(vídeo de respaldo por si falla el directo)

Resultados — vale la pena resolver el NP-hard

Frente a los baselines que sustituyen a la herramienta:

Algoritmo	Felicidad media	Mín	Reglas ✓
CP-SAT	0.81	0.50	82 %
greedy	0.49	0.00	55 %
aleatorio	0.44	0.00	51 %

El piso importa: los baselines dejan a alguien en 0.0; CP-SAT nunca.

La equidad, en acción

Caso **Apollo** — mismo sprint, distinto modo:

- **Nash** → mejor equilibrio (media 0.87, piso 0.60).
- **Hugo**, junior con regla learn-skill: 0.50 → 1.00 al activar la equidad.
- Escalabilidad **honest**a: resuelve hasta ~150 tareas / 20 personas; hay techo en el tamaño XL.

Cómo se construyó

METODOLOGÍA CON IA

85 PRs • 419 commits

- CI en verde por PR • cobertura de test por capa.
- Protocolo **delegar** / **revisar** / **rechazar** — huecos detectados y corregidos.
- Proceso **auditable**, no vibe coding.

Conclusiones

- **H1** (modelado) validada · **H2** (algorítmica) validada con matices · **H3** (metodológica) validada con reservas.
- Un problema **NP-hard** formalizado, resuelto y **hecho usable**.
- Lo más duradero: las decisiones de equidad quedan **explícitas y explicables**, ya no ocultas.

Gracias

Preferencias dentro del modelo,
decisiones de equidad a la vista.

github.com/josec-hdez/SprintWell

¿Preguntas?